Heurísticas y colonia de abejas para el ciclo hamiltoniano

Módulo evolutivo, algoritmo genético y Artificial Bee Colony

Investigación del tema asignado a Integrante 3 | 2 de noviembre de 2026

Alcance. Este documento responde al enunciado proporcionado. No se adjuntó el README ni la práctica original; por ello, los parámetros y la adaptación al ciclo hamiltoniano se presentan como *propuestas didácticas*, no como requisitos confirmados ni como resultados experimentales del FHCP.

1. El problema que se quiere resolver

Un **grafo** $G = (V, E)$ reúne vértices V y aristas E que indican qué pares están conectados. Un **ciclo hamiltoniano** recorre cada vértice exactamente una vez y vuelve al inicial por una arista existente. Si hay n vértices, una ruta puede escribirse $R = (v_1, v_2, \ldots, v_n, v_1)$: el primer vértice aparece de nuevo al final solamente para cerrar el ciclo. La pregunta de decisión es si existe alguna ruta con esas propiedades.[1] No basta con escribir una permutación de vértices: deben existir todas sus aristas consecutivas, incluida (v_n, v_1).

El problema del viajante (TSP) agrega costos a las aristas y busca un ciclo de costo mínimo; el problema hamiltoniano planteado aquí busca *existencia*. Esta distinción cambia la evaluación: una ruta barata no sirve si usa aristas ausentes. El conjunto FHCP es una colección de instancias difíciles del problema hamiltoniano; el ejemplo pequeño de la sección 4 fue inventado para explicar el método y no pertenece a ese conjunto.[2]

Hay que separar también **camino hamiltoniano** y **ciclo hamiltoniano**. Un camino visita todos los vértices sin repetirlos, pero no necesita regresar al punto de partida. Un ciclo exige además una última arista que conecte el final con el inicio. Esa condición de cierre es fácil de olvidar cuando la ruta se guarda como una lista sin repetir el primer vértice al final. Para comprobarla, la implementación debe tratar la lista como circular.

2. Módulo evolutivo y heurísticas

2.1. Qué es una heurística

Una **heurística** es una regla práctica que usa información disponible para orientar una búsqueda hacia opciones prometedoras. Por ejemplo, al extender una ruta parcial puede preferirse un vecino que deje más posibilidades de continuar. Eso ahorra exploración en muchos casos, pero una preferencia no demuestra que exista una solución ni que se haya encontrado la mejor. En búsqueda informada, una función heurística suele estimar el costo o la distancia restante a la meta; en búsqueda de poblaciones también se usan puntuaciones para favorecer candidatos.[3]

Conviene distinguir tres niveles: **representación** (cómo se codifica una posible ruta), **evaluación** (qué tan prometedora parece) y **comprobación** (si satisface exactamente la definición de ciclo). Una heurística guía el trabajo; el comprobador da el veredicto sobre una ruta concreta.

Ejemplo de orientación. Si una ruta parcial termina en C y puede pasar a D o a E, una regla podría preferir el vértice que conserve más conexiones hacia los aún no visitados. Esa regla usa la estructura del grafo para decidir qué probar primero. Puede equivocarse: elegir una continuación que parece buena ahora puede impedir el regreso al inicio después. Por ello, una heurística se evalúa por su utilidad práctica para organizar intentos, no por su capacidad de certificar una respuesta.

En métodos de población aparece otra distinción: **exploración** significa probar regiones nuevas del espacio de rutas, mientras que **explotación** significa mejorar rutas ya prometedoras. Si solo se explota, varias soluciones pueden quedarse cerca del mismo resultado mediocre; si solo se explora, no se aprovecha lo aprendido. La colonia artificial de abejas combina ambos comportamientos mediante observadoras que se concentran en fuentes buenas y exploradoras que renuevan fuentes estancadas.[7,8]

2.2. Cómo representar y modificar una posible solución

Para un grafo con vértices A, B, C, D, E, F, puede guardarse la permutación (A, B, C, D, E, F) e interpretar que después de F se intenta regresar a A. Fijar A al principio evita copias debidas a rotaciones de la misma ruta. En grafos no dirigidos, una ruta y su reverso describen el mismo ciclo; eliminar esa duplicación es opcional.

Una **solución candidata** puede generarse con una permutación aleatoria. Se modifica mediante un *intercambio* de dos posiciones, la inversión de un tramo o la inserción de un vértice en otra posición. Estas operaciones conservan la propiedad de visitar una vez cada vértice, pero pueden crear o destruir aristas válidas. Por eso se vuelve a evaluar y comprobar cada cambio. Para un recorrido parcial, la información útil puede incluir aristas disponibles, vértices sin visitar y posibilidades de cerrar el ciclo. Para una permutación completa, puede incluir el número y la localización de enlaces ausentes.

Una permutación de n vértices ofrece un espacio de búsqueda grande: fijar el inicio deja $(n-1)!$ órdenes posibles. En un grafo no dirigido, leer un ciclo en sentido inverso representa el mismo conjunto de aristas, por lo que incluso entre esos órdenes hay duplicados. La codificación debe preservar una regla sencilla: cada vértice aparece exactamente una vez en la lista y el regreso se agrega al interpretarla. Así, la búsqueda se concentra en mejorar las conexiones en lugar de corregir vértices repetidos.

Cambio	**Ejemplo y efecto sobre** (A, B, C, D, E, F)
Intercambio	Cambiar C y E produce (A, B, E, D, C, F). Se revisan las conexiones alrededor de las dos posiciones.
Inversión	Invertir el tramo C, D, E produce (A, B, E, D, C, F). Se revisan los extremos y la orientación interna del tramo.
Inserción	Mover E antes de C produce (A, B, E, C, D, F). Se revisan las conexiones de la posición original y de la nueva.

Los dos primeros cambios coinciden en este ejemplo corto, pero son operadores distintos en general: el intercambio afecta exactamente dos posiciones, mientras que la inversión puede alterar todo un tramo. En este trabajo el pseudocódigo elige solo *intercambio* para que la adaptación de ABC sea inequívoca.

2.3. Función de evaluación, fitness y validez

Sea $R = (v_1, \ldots, v_n)$ una permutación y definamos $v_{n+1} = v_1$. Una evaluación simple *propuesta para este documento* cuenta aristas inexistentes:

$$b(R) = \sum_{i=1}^{n} \mathbf{1}\big[(v_i, v_{i+1}) \notin E\big], \qquad F(R) = \frac{1}{1 + b(R)}. \tag{1}$$

Aquí b se minimiza; F es el **fitness**, una calidad positiva que se maximiza. Si hay dos enlaces faltantes, $F = 1/3$; si no falta ninguno, $F = 1$. Esta puntuación hace comparables rutas incompletas y permite elegir probabilísticamente entre ellas. No es la única función posible y no implica distancias, pues el problema descrito no da pesos a las aristas.

La **validez** se comprueba de forma explícita: (1) la ruta contiene exactamente los n vértices distintos del grafo; (2) cada par consecutivo es una arista; y (3) también existe la arista del último al primero. Si la representación garantiza una permutación, $b(R) = 0$ equivale a validez para un grafo simple no dirigido, pero conviene conservar un verificador independiente para detectar errores de representación o de implementación. Una ruta con $b = 1$ es *casi* válida, nunca una solución.

La expresión $\mathbf{1}[P]$ vale 1 cuando la proposición P es verdadera y 0 en caso contrario. Por eso $b(R)$ revisa exactamente n conexiones: las $n - 1$ parejas consecutivas de la lista y la conexión de regreso. El valor $F(R)$ siempre es positivo y decrece al aumentar los enlaces ausentes. Se eligió esta transformación porque la selección por ruleta necesita pesos no negativos y porque resulta fácil explicar por qué una fuente con menos fallos recibe más atención.[8]

Ejemplo mínimo de la diferencia. Si una ruta contiene todos los vértices y sus primeras $n - 1$ conexiones existen, pero falta la última arista hacia el inicio, entonces $b = 1$ y $F = 1/2$. El fitness expresa que la ruta está cerca del objetivo según *esta* medida; el verificador responde "inválida". Dos rutas distintas con $b = 1$ reciben igual nota aunque una sea más fácil de reparar. Esta igualdad produce una **meseta** y explica por qué el operador de vecindad y las exploradoras importan tanto como la fórmula de fitness.

2.4. Algoritmo genético (GA) como referencia

Un **algoritmo genético** mantiene una **población** de candidatos, llamados individuos. Cada individuo codifica una ruta. En cada **generación** se evalúan los individuos, se **seleccionan** padres favoreciendo los mejores, se hace **cruce** para combinar información de dos padres y se aplica **mutación** para introducir variaciones. Después se forma una nueva población y se repite. Esta es la estructura general que la práctica menciona como referencia y que el equipo tendrá que conocer antes de programarla.[4,5]

En rutas representadas como permutaciones, un cruce ingenuo puede repetir vértices y omitir otros. Por ejemplo, los padres (B, C, D, E, F) y (C, D, B, E, F), cortados después de dos posiciones y pegados sin cuidado, pueden producir (B, C, B, E, F): B se repite y falta D. Debe usarse un cruce que preserve permutaciones (por ejemplo, uno basado en orden o PMX), o reparar el descendiente.[6] Una mutación por intercambio también preserva la permutación. La selección puede hacerse por torneo o por ruleta; la decisión concreta depende del diseño de GA que adopte el equipo.

El ciclo básico de GA se entiende mejor como una secuencia de responsabilidades:

1. **Inicializar:** crear varias rutas diferentes para no depender de un solo punto de partida.
2. **Evaluar:** asignar fitness a cada individuo; el verificador de validez puede detener la búsqueda si aparece un ciclo.
3. **Seleccionar padres:** dar mayor oportunidad de reproducción a rutas mejor puntuadas, sin tener que copiar únicamente a la mejor.
4. **Cruzar y mutar:** combinar información de padres con un operador que conserve permutaciones y modificar algunas rutas para crear variedad.
5. **Reemplazar:** decidir qué individuos entran en la siguiente generación; opcionalmente conservar al mejor para no perder un avance.
6. **Repetir o detener:** continuar hasta hallar un ciclo o agotar el presupuesto definido para la búsqueda.

La **selección por torneo** compara unos pocos individuos elegidos al azar y toma al mejor del grupo; la **ruleta** asigna probabilidades proporcionales a un fitness positivo. El **cruce** intenta heredar segmentos u órdenes útiles de dos padres. La **mutación** introduce cambios que los padres no aportaron.

Ninguna de estas operaciones, por sí sola, garantiza que las aristas de la ruta final existan: siempre hace falta evaluar y verificar.[4,5,6]

GA y ABC comparten la idea de mantener varias soluciones y puntuarlas. Su mecánica principal difiere: GA recombina padres y muta descendientes; ABC asigna una fuente de alimento a cada abeja empleada, mejora fuentes por vecindad, recluta observadoras según calidad y reemplaza fuentes estancadas.[4,5,7]

3. Colonia artificial de abejas: versión elegida

3.1. Origen y alcance de la adaptación

La versión de referencia es **Artificial Bee Colony (ABC)** de Derviş Karaboğa (2005). Su propuesta original trataba optimización numérica: cada fuente de alimento era un vector de números reales. Aquí se conserva *su ciclo de fases* y se propone una adaptación discreta por intercambio para permutaciones de vértices. No se usa la fórmula continua de perturbación de coordenadas sobre rutas, porque produciría valores que no son permutaciones. Tampoco se mezclan mecanismos de otros algoritmos de abejas. Existen trabajos académicos que adaptan ABC a problemas combinatorios y a recorridos, lo que respalda la necesidad de definir operadores discretos; el operador y el fitness exactos de este documento son decisiones didácticas.[7,8,9]

Término ABC	**Interpretación en esta propuesta**
Fuente de alimento	Una permutación de todos los vértices, con regreso implícito al inicial.
Néctar o calidad	El fitness $F(R)$ de la ecuación (1).
Abeja empleada	Trabaja sobre una fuente asignada e intenta mejorarla mediante un intercambio.
Abeja observadora	Elige una fuente con mayor probabilidad cuanto mayor sea su fitness; también intenta mejorarla.
Abeja exploradora	Sustituye una fuente que acumula demasiados intentos sin mejora por una nueva permutación.

3.2. Qué ocurre en un ciclo de ABC

En la **inicialización** se crean fuentes independientes y se calcula su calidad. La analogía con una fuente de alimento no significa que haya abejas moviéndose físicamente por el grafo: cada fuente es una estructura de datos que describe una ruta. Se asocia una abeja empleada a cada fuente. En la versión original de ABC, las abejas empleadas y las observadoras forman los dos grupos principales, y una empleada puede convertirse en exploradora cuando su fuente se abandona.[7]

Durante la **fase de empleadas**, cada fuente recibe al menos un intento local. En la adaptación propuesta, la empleada intercambia dos vértices de su ruta y compara el fitness nuevo con el anterior. Si el intento mejora, conserva la vecina. Si no mejora, la fuente permanece igual y el contador registra el fracaso. Esta fase asegura que incluso una fuente poco popular tenga oportunidad de mejorar.

En la **fase de observadoras**, se publica la calidad de las fuentes y se eligen con las probabilidades de la ecuación (2). Una fuente de alta calidad puede recibir varias observadoras en el mismo ciclo y otra puede no recibir ninguna. Cada observadora intenta su propio cambio local y aplica la misma decisión voraz. Aquí aparece la explotación: se invierten más intentos en rutas que ya parecen prometedoras.[8]

Por último, la **fase de exploradoras** revisa el estancamiento. Una fuente que acumuló `limit` intentos sin mejora deja de ocupar un lugar de la colonia y se reemplaza por otra ruta creada desde

cero. Esto reintroduce diversidad. El algoritmo guarda la mejor solución histórica aparte, porque abandonar la fuente actual no debería borrar un resultado útil anterior. El ciclo completo se repite hasta hallar un ciclo válido o agotar el presupuesto de búsqueda.[7,8]

3.3. Selección y abandono

Si existen SN fuentes y todas tienen fitness positivo, la probabilidad propuesta de que una observadora elija la fuente i es

$$p_i = \frac{F(R_i)}{\sum_{j=1}^{SN} F(R_j)}. \tag{2}$$

Una fuente buena tiene más oportunidades, pero una de menor calidad conserva probabilidad no nula. Después de crear una vecina R', se usa **selección voraz**: se reemplaza R solo si $F(R') > F(R)$; en caso contrario se conserva R y aumenta su contador de intentos sin mejora. En esta propuesta, un empate también cuenta como intento sin mejora. Si el contador alcanza `limit`, la fuente se abandona y una exploradora crea una nueva permutación; su contador vuelve a cero. Una mejora también reinicia el contador. Se guarda aparte la mejor ruta vista hasta el momento para no perderla al reemplazar fuentes. El parámetro `limit` y estas fases provienen del ABC de referencia; los detalles de empate y operador discreto son decisiones explícitas de la adaptación.[7,8]

3.4. Pseudocódigo de la adaptación propuesta

1. Recibir el grafo $G = (V, E)$, el número de fuentes SN, el número máximo de ciclos $C_{\text{máx}}$ y `limit`. Usar SN empleadas y SN observadoras.
2. Crear SN permutaciones aleatorias de V con el vértice inicial fijo; para cada una calcular b y F, y poner su contador en cero. Guardar la mejor encontrada.
3. Si alguna permutación pasa la comprobación exacta de ciclo hamiltoniano, devolverla.
4. Para cada **abeja empleada**: elegir dos posiciones distintas, excepto la inicial, e intercambiar sus vértices. Evaluar la vecina. Si mejora estrictamente, reemplazar la fuente, reiniciar su contador, actualizar la mejor y comprobar si ya es un ciclo; en otro caso, aumentar el contador.
5. Calcular las probabilidades p_i con la ecuación (2).
6. Para cada **abeja observadora**: elegir una fuente según p_i, crear otra vecina mediante intercambio y aplicar la misma regla voraz, el mismo contador y la comprobación de ciclo al aceptar una mejora.
7. Para cada fuente cuyo contador haya alcanzado `limit`: activar la fase **exploradora**, sustituirla por una nueva permutación aleatoria, evaluarla, reiniciar su contador y comprobar si ya es un ciclo.
8. Actualizar la mejor ruta vista y comprobarla con el verificador exacto. Si es válida, devolverla.
9. Repetir los pasos 4 a 8 hasta $C_{\text{máx}}$ o hasta otro límite de recursos acordado.
10. Si no se encontró un ciclo, informar *"no encontrado con este presupuesto de búsqueda"*. El agotamiento del tiempo de una heurística no demuestra que el grafo carezca de ciclo.

Los ajustes principales son SN (cuántas regiones se exploran en paralelo), $C_{\text{máx}}$ (presupuesto de búsqueda), `limit` (cuándo abandonar una fuente estancada) y el **operador de vecindad** (qué cambios puede intentar cada abeja). Una población muy pequeña puede perder diversidad; una muy grande cuesta más evaluaciones. Un `limit` bajo promueve exploración frecuente; uno alto permite insistir más sobre una fuente. No hay valores universales: se ajustarían después con experimentos reproducibles, fuera del alcance de esta investigación.[7,8]

Ajuste	Si se reduce	Si se aumenta
SN	Menos fuentes y menos diversidad inicial; cada ciclo cuesta menos.	Más rutas simultáneas y más evaluaciones por ciclo.
$C_{\text{máx}}$	La búsqueda acaba antes, incluso si estaba progresando.	Se permiten más intentos, con mayor costo total.
`limit`	Las fuentes se abandonan con rapidez y se explora más.	Se insiste más tiempo en cada fuente, incluso si está estancada.
Vecindad	Un cambio muy local es barato, pero puede avanzar lentamente.	Cambios más amplios recorren más posibilidades, aunque pueden destruir conexiones útiles.

Cada evaluación de $b(R)$ puede revisar las n parejas de la ruta una vez. Si las aristas del grafo se consultan rápidamente, ese recuento crece linealmente con n. Un intercambio solo altera las conexiones que tocan las dos posiciones cambiadas, de modo que una implementación posterior podría actualizar el valor localmente; para comprender y verificar el método resulta más transparente recalcular las n parejas. Esto es una observación de diseño, no un requisito de la práctica.

4. Ejemplo didáctico de seis vértices

Tomemos $V = \{A, B, C, D, E, F\}$ y el siguiente grafo no dirigido, definido por su lista completa de aristas:

$$E = \{AB, BC, CD, DE, EF, FA, AC, BD, CE, DF\}.$$

El orden de letras en una arista no importa: AB y BA denotan la misma conexión. Toda ruta de la tabla empieza en A y muestra el regreso final a A.

Fuente	Ruta candidata	Enlaces ausentes	F
R_1	$A-B-C-D-F-E-A$	EA	$1/2$
R_2	$A-D-C-B-E-F-A$	AD, BE	$1/3$
R_3	$A-D-B-E-C-F-A$	AD, BE, CF	$1/4$

Paso 1: evaluar y elegir. Las tres rutas visitan los seis vértices una vez antes de volver a A, pero ninguna es todavía válida. Por la ecuación (2), la suma de fitness es $1/2 + 1/3 + 1/4 = 13/12$. Sus probabilidades son $p_1 = 6/13$, $p_2 = 4/13$ y $p_3 = 3/13$. Por tanto, una observadora prefiere R_1, aunque podría escoger cualquiera.

Paso 2: intento de empleada. La empleada de R_1 intercambia C y D y obtiene $A - B - D - C - F - E - A$. Faltan CF y EA, así que $b = 2$ y $F = 1/3$. Como empeora respecto de $1/2$, se rechaza y el contador de R_1 aumenta en uno.

Paso 3: intento de observadora. En una realización posible, una observadora selecciona R_1 y luego intercambia F y E. Obtiene $R' = A - B - C - D - E - F - A$. Ahora $b(R') = 0$ y $F(R') = 1$, por lo que se acepta y el contador vuelve a cero.

Paso 4: comprobación independiente. Los vértices antes del regreso son A, B, C, D, E, F, todos distintos y todos los de V. Se revisan las seis aristas: $AB \in E$, $BC \in E$, $CD \in E$, $DE \in E$, $EF \in E$ y, crucialmente, $FA \in E$. Por eso R' sí es un ciclo hamiltoniano. Si faltara FA, sería solo un camino que visita todos los vértices.

Posición	1	2	3	4	5	6
Conexión	$A \to B$	$B \to C$	$C \to D$	$D \to E$	$E \to F$	$F \to A$
¿Existe?	Sí	Sí	Sí	Sí	Sí	Sí

La última columna demuestra el cierre. En una representación sin el A final, esa columna debe generarse de manera explícita al verificar. También se comprueba que la lista contiene seis nombres distintos, no solo que aparecen seis posiciones: una lista como (A, B, C, D, E, E) tendría la longitud correcta, pero repetiría E y omitiría F.

Paso 5: abandono, si no se encontrara una solución. Para ilustrar la fase exploradora por separado, supongamos `limit`$= 2$ y que R_2 prueba dos vecinos sin mejora. Por ejemplo, al intercambiar E y F produce $A{-}D{-}C{-}B{-}F{-}E{-}A$, con $b = 3$; al intercambiar B y E produce $A{-}D{-}C{-}E{-}B{-}F{-}A$, también con $b = 3$. Ambos se rechazan. El contador llega a dos y se reemplaza R_2 por una permutación nueva. Esta rama es hipotética: si se ejecuta el paso 3 y se encuentra R', el algoritmo ya puede detenerse.

Carácter del ejemplo. Las elecciones de intercambios y la selección de R_1 son una secuencia ilustrativa posible, no una ejecución medida ni un resultado del FHCP.

4.1. Qué enseña el ejemplo

El paso de R_1 a R' muestra que una modificación pequeña puede reparar la única arista faltante, pero el primer intento de la empleada también muestra que un intercambio puede empeorar la ruta. La regla voraz evita aceptar esa pérdida en la fuente actual. Las probabilidades no obligan a escoger la mejor fuente: solo hacen que reciba más atención en promedio. Por último, `limit` se mide en intentos fallidos de una fuente, no en la edad total de la colonia. Estas cuatro ideas son las que conviene transmitir al equipo al explicar ABC.

5. Aplicación al ciclo hamiltoniano: opciones y dificultades

Opción A: permutaciones completas con penalización. Es la utilizada aquí. Su ventaja es que cada candidato conserva todos los vértices exactamente una vez; se mide cuántas aristas faltan y se busca $b = 0$. Su dificultad es que un solo intercambio puede alterar hasta cuatro enlaces y una meseta de rutas con el mismo b puede estancar la búsqueda.

Opción B: construir caminos válidos de forma incremental. Cada paso añade solo un vecino unido por una arista y no visitado. Así se evitan enlaces inválidos durante la construcción, pero el camino puede quedar bloqueado antes de incluir todos los vértices o sin arista de regreso. Para usar ABC habría que definir con cuidado qué es una fuente, cómo se modifica y cómo se reinicia.

Opción C: operadores o reparación guiados por el grafo. Se pueden preferir intercambios que reparen enlaces ausentes o usar inversión de segmentos y búsqueda local. Es una extensión plausible porque hay adaptaciones combinatorias de ABC para recorridos, pero requiere especificar y evaluar su sesgo y costo computacional.[9]

Dificultades comunes. La calidad parcial no garantiza que un ciclo exista; los óptimos locales y mesetas pueden hacer que la búsqueda repita intentos; las rutas equivalentes por rotación o reverso desperdician evaluaciones; y el grafo puede tener restricciones estructurales que hagan imposible la meta. Un verificador exacto permite confirmar una solución encontrada, pero una búsqueda heurística que se agota no certifica la inexistencia del ciclo. Si la instancia usa aristas dirigidas o un formato concreto, deben ajustarse representación y verificación.

5.1. Criterios para interpretar un resultado

Un ciclo producido por el método se puede revisar con un procedimiento determinista: contar vértices, comprobar que no se repiten, consultar cada pareja consecutiva y consultar la arista de cierre. Esa revisión sí confirma un resultado positivo. En cambio, si termina el presupuesto sin ruta válida, la conclusión correcta es que *esta ejecución no encontró un ciclo*. Puede haber uno en otra

región del espacio de búsqueda. La calidad de la mejor ruta encontrada, el número de intentos y el presupuesto utilizado describen el proceso, pero no convierten una búsqueda heurística en una prueba de inexistencia.

Para comparar una futura implementación de ABC con GA de manera justa, ambos tendrían que resolver la misma instancia y usar el mismo verificador de ciclos. El fitness podría compartirse para facilitar la interpretación de resultados; si se usa una función distinta, habría que explicarla. El documento no reporta tiempos, tasas de éxito ni comparaciones, porque esas cifras exigirían ejecuciones reales y un protocolo de medición.

6. Fuentes consultadas y qué respaldan

[1] **Flinders Hamiltonian Cycle Project**, "Flinders Hamiltonian Cycle Project", s. f., https://sites.flinders.edu.au/flinders-hamiltonian-cycle-project/. Define el problema hamiltoniano y distingue su objetivo del TSP. El equipo del proyecto, "FHCP Challenge Set", s. f., https://sites.flinders.edu.au/flinders-hamiltonian-cycle-project/fhcp-challenge-set/, describe el conjunto FHCP y su formato. Se citan como fuente institucional de contexto, no como evidencia experimental de este ejemplo.

[2] **Michael Haythorpe**, "FHCP Challenge Set: The First Set of Structurally Difficult Instances of the Hamiltonian Cycle Problem", 2018, https://arxiv.org/abs/1902.10352. Respalda la identidad y finalidad del conjunto FHCP. La fecha de publicación del artículo de revista es 2018; la versión arXiv enlazada es posterior.

[3] **David L. Poole y Alan K. Mackworth**, *Artificial Intelligence: Foundations of Computational Agents*, 3.ª ed., Cambridge University Press, 2023, sección 3.6, https://www.artint.info/3e/html/ArtInt3e.Ch3.S6.html. Respalda la explicación de búsqueda informada y de función heurística como guía estimativa.

[4] **John H. Holland**, *Adaptation in Natural and Artificial Systems*, 1975; reimpresión MIT Press, 1992, https://doi.org/10.7551/MITPRESS/1090.001.0001. Fundamento histórico del algoritmo genético y de la adaptación mediante recombinación y selección.

[5] **Xin-She Yang**, "Metaheuristic Optimization", *Scholarpedia*, 2011, https://www.scholarpedia.org/article/Metaheuristic_Optimization. Respalda el esquema general de GA: codificación, fitness, población, selección, cruce, mutación y reemplazo.

[6] **David E. Goldberg y Robert Lingle Jr.**, "Alleles, Loci, and the Traveling Salesman Problem", *Proceedings of the First International Conference on Genetic Algorithms*, 1985, https://gpbib.pmacs.upenn.edu/icga/icga1985-ocr.pdf. Respalda el uso de permutaciones y cruces especializados, como PMX, para recorridos.

[7] **Derviş Karaboğa**, "An Idea Based on Honey Bee Swarm for Numerical Optimization", informe técnico TR06, Universidad de Erciyes, 2005, enlace al informe. Fuente primaria de las fases de ABC, roles de abejas, fuentes, selección y parámetro `limit`.

[8] **Derviş Karaboğa**, "Artificial Bee Colony Algorithm", *Scholarpedia* 5(3):6915, 2010, enlace al artículo. Expone la versión de ABC con selección probabilística, mejora voraz, abandono y controles principales.

[9] **Indadul Khan y Manas Kumar Maiti**, "A Swap Sequence Based Artificial Bee Colony Algorithm for Traveling Salesman Problem", *Swarm and Evolutionary Computation* 44, 428–438, 2019, https://doi.org/10.1016/j.swevo.2018.05.006. Respalda que las variantes discretas para recorridos requieren operadores de permutación; no se atribuye a este artículo la adaptación específica propuesta aquí.